

Project Work – To Do List Application

Realizzare una semplice applicazione per la gestione di una To Do List, pubblicata su Azure e gestita tramite repository GitHub e pipeline di deploy.

L'obiettivo del project work non è valutare la complessità dell'applicazione o la perfezione del codice, ma comprendere come affronti un problema applicativo end-to-end, quali scelte architetturali fai, come le motivi e come organizzi il lavoro in un contesto parzialmente nuovo

Scenario

Si vuole realizzare una piccola applicazione per gestire attività personali o di team.

L'applicazione dovrà essere utilizzabile inizialmente da browser tramite una web application.

In prospettiva futura, si potrebbe voler rendere disponibili le stesse funzionalità anche ad altri client, ad esempio una mobile app nativa iOS / Android o altri sistemi esterni.

Nel project work ti chiediamo quindi di ragionare su quale architettura applicativa ritieni più adatta, tenendo conto sia dell'esigenza attuale sia delle possibili evoluzioni future.

Funzionalità richieste

L'applicazione deve consentire le operazioni base su una To Do List:

1. visualizzare l'elenco delle attività;
2. creare una nuova attività;
3. modificare un'attività esistente;
4. completare o riaprire un'attività;
5. eliminare un'attività.

Ogni attività può contenere, ad esempio:

- identificativo;
- titolo;
- descrizione opzionale;
- stato;
- data di creazione;
- eventuale data di completamento

- eventuale data di completamento.

Non è richiesta una UI particolarmente elaborata. È sufficiente che l'applicazione sia chiara, semplice e utilizzabile.

Requisiti tecnici generali

La soluzione dovrà prevedere:

- una web application sviluppata preferibilmente con **ReactJS** e **C#**;
- una forma di persistenza dei dati;
- pubblicazione su **Microsoft Azure**;
- codice sorgente versionato su **GitHub**;
- pipeline di build e/o deploy tramite **GitHub Actions**;
- documentazione delle scelte effettuate.

La scelta dello stack applicativo, del modello architetturale e delle risorse cloud è lasciata a te per cui non è un problema nel caso differisca da quanto sopra

Aspetti architetturali da valutare

Ti chiediamo di scegliere liberamente l'architettura che ritieni più adeguata al contesto.

È però importante che nel deliverable tu spieghi:

- quale architettura hai scelto;
- perché l'hai scelta;
- quali alternative hai considerato se l'hai fatto;
- quali vantaggi offre la tua scelta ed eventualmente quali limiti o compromessi introduce;
- come la soluzione potrebbe evolvere se in futuro fosse necessario esporre le funzionalità anche ad altri client.

Non è importante la forma, quanto il contenuto :-)

Requisiti cloud

La soluzione deve essere deployata su **Microsoft Azure**.

La scelta delle risorse Azure è parte integrante del project work.

Ti chiediamo quindi di individuare autonomamente quali servizi utilizzare per:

- ospitare l'applicazione;
- gestire la persistenza;
- supportare il deploy;

Non è richiesto l'utilizzo di servizi complessi o architetture enterprise.

È preferibile una soluzione semplice, comprensibile e proporzionata allo scopo del project work.

Nel deliverable dovrai motivare le risorse Azure scelte, indicando anche eventuali alternative che avresti potuto utilizzare.

Requisiti DevOps

Il codice deve essere pubblicato su un repository GitHub.

Il repository dovrebbe contenere una struttura chiara e leggibile, ad esempio con separazione tra codice applicativo, configurazioni e documentazione, se coerente con la soluzione scelta.

È richiesta almeno una pipeline GitHub Actions per automatizzare una parte significativa del processo, ad esempio:

- build;
- test, se presenti;
- deploy;

Non è necessario realizzare una pipeline enterprise completa.

È però importante che sia chiaro:

- cosa fa la pipeline;
- quando viene eseguita;
- quali prerequisiti richiede;
- quali eventuali limiti ha.

Deliverable richiesti

Al termine del project work dovranno essere consegnati i seguenti deliverable.

1. Repository GitHub

Il repository dovrà contenere:

- codice sorgente della soluzione;
 - configurazione delle pipeline GitHub Actions;
 - eventuali file di configurazione necessari;
 - documentazione minima per eseguire e comprendere il progetto.
-

2. Applicazione funzionante

Dovrà essere disponibile una versione deployata su Azure, oppure dovranno essere fornite istruzioni chiare per eseguire il deploy.

L'applicazione dovrà consentire almeno le operazioni CRUD principali sulla To Do List.

Sono accettabili piccoli bug o limiti implementativi, purché siano dichiarati.

3. Schema architetturale

Dovrà essere prodotto un semplice schema architetturale che rappresenti:

- le principali componenti applicative;
- il flusso tra utente, applicazione e persistenza;
- le risorse Azure utilizzate;
- il processo di build/deploy tramite GitHub Actions.

Lo schema può essere realizzato con lo strumento preferito, ad esempio:

- immagine;
 - diagramma;
 - Markdown;
 - Mermaid;
 - draw.io;
 - altro strumento a scelta.
-

4. Spiegazione delle scelte

Nel README o in un documento dedicato dovranno essere spiegati:

- architettura scelta;
- motivazione della scelta;
- risorse Azure utilizzate;

- risorse Azure utilizzate;
- motivazione delle risorse Azure;
- alternative considerate;
- gestione della persistenza;
- organizzazione del repository;
- funzionamento della pipeline;
- limiti noti;
- possibili evoluzioni future.

Aspetti non prioritari

Non sono prioritari:

- interfaccia grafica avanzata;
- autenticazione utenti;
- gestione ruoli;
- autorizzazioni complesse;
- test completi;
- logging avanzato;
- osservabilità completa;
- infrastruttura enterprise;
- architettura a microservizi;

Se decidi di introdurre uno di questi aspetti, ti chiediamo di spiegare perché lo ritieni utile rispetto allo scope del project work.

Aspetti che verranno osservati

La valutazione si concentrerà principalmente su:

Area	Cosa verrà osservato
Ragionamento architetturale	Capacità di scegliere e motivare una soluzione coerente
Comprensione del contesto	Capacità di considerare esigenze attuali ed evoluzioni future
Pragmatismo	Capacità di mantenere la soluzione semplice e proporzionata
Cloud awareness	Capacità di individuare risorse Azure adeguate allo scenario
DevOps mindset	Uso ragionato di GitHub, versionamento e pipeline
...	...

Documentazione	Chiarezza nel descrivere scelte, limiti e alternative
Autonomia	Capacità di documentarsi e procedere in modo indipendente
Problem solving	Capacità di gestire ostacoli, compromessi e workaround

Cosa non verrà valutato in modo predominante

Non verrà valutata in modo predominante la perfezione del codice.

Piccoli bug o incompletezze sono accettabili, se:

- non impediscono di comprendere l'impostazione della soluzione;
 - sono dichiarati;
 - è chiaro come potrebbero essere corretti in una fase successiva.
-

Nota finale

L'obiettivo del project work è osservare il tuo approccio al problema.

Non ci interessa solo vedere l'applicazione funzionare, ma capire:

- come analizzi il requisito;
- quali scelte fai;
- come valuti le alternative;
- come mantieni semplice la soluzione;
- come documenti ciò che hai fatto;
- come ragioni su una possibile evoluzione futura.

La soluzione può essere semplice purché sia chiara, motivata e coerente.